📸 Reproduce & Screenshot Guide

Practical, run-based guide: **where each bug is, how to make it appear on screen, and exactly what to capture.** All findings come from actually running the project (no mock images). Take your own screenshots at each "📸 Capture" step.

> Two tracks: **A) Web app — runnable right now** (real console errors you can screenshot today) and **B) Mobile app — needs an Android emulator/device** (steps to reproduce each screen).

A) WEB APP — runnable today (`enatega-multivendor-web`, Next.js)

A0. Start it

```
cd enatega-multivendor-web
npm install --legacy-peer-deps
NEXT_PUBLIC_SERVER_URL="https://aws-server-v2.enatega.com/" \
NEXT_PUBLIC_WS_SERVER_URL="wss://aws-server-v2.enatega.com/" \
npx next dev --webpack
# open http://localhost:3000
```

✅ The landing page renders (proof: `01-manual-testing/web-run-screenshots/web-home.png`). The **real console errors below print in the terminal AND in the browser DevTools console.**

WEB-1 — Apollo `connectToDevTools` deprecated

- **File:** `enatega-multivendor-web/lib/hooks/useSetApollo.tsx:218`
- **What appears:** `An error occurred! … ApolloClient | connectToDevTools | Please use` devtools.enabled `instead.`
- **How to see:** as soon as any page loads (terminal running `npm run dev`, or browser DevTools → Console).
- 📸 **Capture:** the terminal / console line showing the `connectToDevTools` message.

WEB-2 — `useLazyQuery` deprecated options (`onCompleted` / `onError` / `variables`)

- **File:** `enatega-multivendor-web/lib/hooks/useLazyQueryQL.tsx:37-41` (used in **13** places app-wide)
- **What appears (3 distinct, repeated on every render):**
- `useLazyQuery | onCompleted | … switch to derived state using data …`
- `useLazyQuery | onError | … switch to derived state …`
- `useLazyQuery | variables | Pass all variables to the returned execute function instead.`
- **How to see:** open the home page; the console floods with `An error occurred!` (15× per load in this run).
- 📸 **Capture:** the console block showing the repeated `useLazyQuery` messages.
- 🧾 Full decoded text: `01-manual-testing/web-run-screenshots/CONSOLE-ERRORS-live.txt`

WEB-3 — next-intl webpack parsing warning

- **File:** `enatega-multivendor-web/next.config.mjs:1` (`next-intl/plugin`)
- **What appears:** `Parsing of …/next-intl/…/extractor/format/index.js for build dependencies failed at 'import(t)'`
- **How to see:** in the `npm run dev` startup output.

- 📸 **Capture:** the startup terminal line with the `next-intl … failed at 'import(t)'` warning.

B) MOBILE APP — needs Android emulator/device (`enatega-multivendor-app`, Expo)

B0. Start it

```
cd enatega-multivendor-app
npm install
npx expo start           # press "a" for Android emulator, or scan in Expo Go
# demo login: demo-customer@enatega.com / 123123
```

MOB-1 — Login rejects valid modern-TLD emails

- **File:** `src/screens/Login/useLogin.js:80` (regex `(\.\w{2,3})+$`)
- **Steps:** Login → type `user@company.store` → **Continue**.
- **You'll see:** "Please enter a valid email" (valid address wrongly rejected).
- 📸 **Capture:** the email field with the red error message.

MOB-2 — Password "eye" icon inverted

- **File:** `src/screens/Login/useLogin.js:33` + `src/screens/Login/Login.js:88-89`
- **Steps:** enter a registered email → **Continue** → look at the password field + eye icon.
- **You'll see:** field is masked but the icon already shows `eye-slash` (points the wrong way).
- 📸 **Capture:** the password field showing the mismatched eye icon.

MOB-3 — No `testID` / accessibility labels (automation + TalkBack)

- **File:** `src/screens/Login/Login.js:69, 88, 117` (25 `TextInput`, **0** `testID` app-wide)
- **Steps:** enable TalkBack, or run `adb shell uiautomator dump` and open the dump.
- **You'll see:** buttons announced "unlabeled"; empty `resource-id` / `content-desc`.
- 📸 **Capture:** TalkBack focus on a button, or the uiautomator dump with empty ids.

MOB-4 — Demo password hard-coded in the client

- **File:** `src/screens/Login/useLogin.js:63-64`
- **Steps:** enter `demo-customer@enatega.com` → **Continue**.
- **You'll see:** password auto-fills and login works without typing a password.
- 📸 **Capture:** the pre-filled password step / successful login.

MOB-5 — Cart quantity has no upper cap

- **File:** `src/context/User.js:114 (addQuantity)`
- **Steps:** open an item → press **+** many times (e.g. 99+) → open cart.
- **You'll see:** quantity keeps growing, line total multiplies without limit.
- 📸 **Capture:** the cart line with the huge quantity/total.

MOB-6 — `deleteItem` mutates cart state in place

- **File:** `src/context/User.js:126` (`cart.splice(...)`)
- **Steps:** add 2 items → delete one quickly → watch the count/list.
- **You'll see:** occasional stale count / item lingering until re-render.
- 📸 **Capture:** the cart showing the inconsistent count.

MOB-7 — Offer filters empty the restaurant list

- **File:** `src/screens/Menu/Menu.js` (reads `freeDelivery` / `acceptVouchers`; backend never sets them — see BACK-1)
- **Steps:** restaurant list → **Offers** → **Free Delivery** (or **Accept Vouchers**) → Apply.
- **You'll see:** the list becomes empty.
- 📸 **Capture:** the empty list after applying the filter.

C) BACKEND / API — no UI needed (screenshot Postman)

BACK-1 — Offer flags never set (root cause of MOB-7)

- **Where:** `aws-server-v2.enatega.com/graphql` — `freeDelivery` / `acceptVouchers` are `false` / `null` for every restaurant. Analysis: `enatega-multivendor-app/QUAL-012_BACKEND_INSTRUCTIONS.md`.

BACK-2 / BACK-3 — Auth contract & missing REST route

- Open `03-api-testing/results/newman-live-report.html` (or run the Postman collection).
- 📸 **Capture:** the green run summary (`postman-live-run.png` already provided) and any request's Test Results.
- Findings: spec `POST /api/v1/auth/login` → **404** on the real backend (it uses GraphQL); nullable `$type` → `GRAPHQL_VALIDATION_FAILED`; protected query with no token → `Unauthorized`.

Index of the real evidence already captured for you

File	Shows
`01-manual-testing/web-run-screenshots/web-home.png`	Web app actually running
`01-manual-testing/web-run-screenshots/CONSOLE-ERRORS-live.txt`	Decoded real console errors (WEB-1/2/3)
`03-api-testing/screenshots/postman-live-run.png`	Live API run — 5 req / 10 assertions / 0 fail
`BUG-LOCATION-MAP.md`	All findings by layer with `file:line`